Trabajo Práctico — Unidad 4: Forma Normal de Boyce-Codd y Desnormalización Controlada en Food Store

Índice

Unidad 4: Normalización Avanzada y Redundancia Proyecto integrador de referencia: Food Store

1. Presentación del trabajo

Este trabajo práctico exige aplicar, sobre una situación nueva del dominio Food Store, las dos herramientas centrales de la Unidad 4: llevar un esquema hasta la **Forma Normal de Boyce-Codd (FNBC)** cuando una dependencia funcional lo exige, y **desnormalizar de manera deliberada y controlada** cuando una consulta real del sistema lo justifica por rendimiento.

El trabajo se organiza en dos partes independientes entre sí, pero unidas por el mismo criterio profesional: ninguna decisión de diseño —normalizar o desnormalizar— se toma por intuición. Se toma analizando dependencias funcionales con rigor, en la Parte 1, y midiendo con EXPLAIN ANALYZE, en la Parte 2.

2. Objetivos de aprendizaje

Al finalizar este trabajo, el estudiante deberá ser capaz de:

1. Identificar dependencias funcionales a partir de una regla de negocio redactada en lenguaje natural.
2. Calcular clausuras de atributos y determinar el conjunto completo de claves candidatas de un esquema.
3. Determinar, con la definición formal de FNBC, si un esquema cumple o no esa forma normal, y aplicar el algoritmo de descomposición sin pérdida cuando no la cumple.
4. Reconocer, en una consulta real, cuándo el costo de un recorrido o de sucesivas reuniones (JOIN) justifica una desnormalización controlada.
5. Escribir al menos una consulta que explote una estructura desnormalizada, y compararla contra su equivalente normalizada mediante EXPLAIN ANALYZE.

3. Consigna general y modo de trabajo

El trabajo se resuelve sobre una copia de la base Food Store ya poblada. La Parte 1 introduce una extensión puntual del dominio, que el estudiante debe crear y poblar con una instancia de ejemplo suficiente para exhibir el problema. La Parte 2 se resuelve directamente sobre el esquema ya existente del proyecto (tablas pedido, detalle_pedido, producto, categoria).

Toda afirmación técnica debe estar acompañada del artefacto SQL o de la salida de EXPLAIN ANALYZE que la demuestra: no se acepta una respuesta exclusivamente narrativa.

4. Parte 1 — Análisis y aplicación de la FNBC: `ControlLoteAlmacen`

4.1. Contexto de dominio

Food Store extiende su operación hacia la distribución mayorista: incorpora **depósitos** propios y recibe de sus proveedores **lotes** de mercadería, que deben pasar un control de calidad antes de

ingresar al inventario. Cuando un lote es de gran volumen, se fracciona y se controla simultáneamente desde más de un depósito. Por cada depósito que interviene en el control de un lote, se designa un **responsable de control** a cargo de esa fracción. La regla de negocio, relevada con el área de logística, es la siguiente:

- Para un lote y un depósito interviniente dados, el responsable de control queda unívocamente determinado.
- Cada responsable de control pertenece, como dato maestro de la dotación de personal, a un único depósito: no controla lotes coordinados desde depósitos distintos.

Este relevamiento se traduce en el siguiente esquema inicial, ya cargado con una instancia de ejemplo. Se asume que las tablas maestras lote y deposito de esta extensión mayorista ya existen (de manera análoga a como sucursal se asumió existente al tratar el caso `AsignacionEntrega`):

```
CREATE TABLE control_lote_almacen (
lote_id                 BIGINT NOT NULL REFERENCES lote(id),
deposito_id             BIGINT NOT NULL REFERENCES deposito(id),
responsable_control_id  BIGINT NOT NULL REFERENCES usuario(id),
PRIMARY KEY (lote_id, deposito_id) );

INSERT INTO control_lote_almacen VALUES
    (501, 30, 801),
    (502, 30, 801),
    (503, 31, 802);
```

4.2. Se pide

a) Enunciar, en notación formal, las dependencias funcionales que se desprenden de la regla de negocio, usando los nombres de atributo `LoteID`, `DepositoID` y `ResponsableControlID`.

LoteID = L, DepositoID = D y ResponsableControlID = R

DF1: {L, D} -> R

DF2: {R} -> D

b) Calcular la clausura de los subconjuntos de atributos que resulten candidatos razonables y determinar el conjunto **completo** de claves candidatas del esquema. Indicar qué atributos son primos y cuáles no.

1. Clausura de {L y D}: Al aplicar dicha regla descubrimos automáticamente al Responsable del control (R). Como con el Lote y el Depósito solos no deducimos nada más, confirmamos que {L,D} es una clave candidata.

2. Clausura de {R}: Al aplicar dicha regla sólo podemos deducir el Depósito (D) al que pertenece. De esta manera, nos damos cuenta que con {R} solo no podemos completar los tres atributos del universo, por lo que no alcanza para que pueda ser una clave
3. Clausura de {L y R}: Al aplicar dicha regla podemos deducir al Depósito (ya que el Responsable determina al Depósito). Como vimos anteriormente, con el Lote solo no logramos determinar nada, y con el Responsable solo logramos determinar al Depósito, por lo cual, es una clave candidata.

Atributos primos: En este caso los tres atributos son primos (Lote, Depósito y Responsable) ya que forman parte de una clave candidata.

c) Determinar, con la definición formal de FNBC (no con la de 3FN), si `control_lote_almacen` cumple o no la forma normal. Justificar identificando explícitamente la dependencia funcional violatoria, si la hubiera, y explicando por qué su determinante no es superclave.

control_lote_almacen no cumple con la definición formal de FNBC, ya que el Responsable del Control o {R} es el determinante del Depósito o {D} ,y a su vez, esta no es una superclave.

{R} no se considera una superclave ya que de por sí solo no logra determinar a todos los atributos del universo, sólo al Depósito.

d) Con la instancia de ejemplo provista, redactar en un párrafo cada una de las tres anomalías clásicas (inserción, borrado, actualización) que la violación detectada habilita, describiendo un escenario concreto sobre esos datos.

1. Inserción: Si la empresa quiere contratar un responsable 803 y asignarlo al depósito 32, si o si primero deberíamos agregar un lote real a dicho depósito. Esto se debe a que LoteID pertenece a la clave primaria y no puede ser nulo.
2. Borrado: Si quisiéramos eliminar el lote 503, el cual es el único operado por el depósito 31, perdemos colateralmente el único registro que indica que el responsable 802 trabaja en el depósito 31. Perdemos un dato maestro.
3. Actualización: Si quisiéramos transferir el responsable 801 (el cual controla los lotes 501 y 502) al depósito 33, deberíamos cambiar dichas filas. Y si por error solo modificamos una, la base queda inconsistente.

e) Aplicar el algoritmo de descomposición sin pérdida visto en clase y entregar el script SQL completo de las tablas resultantes, incluyendo sus claves primarias, sus restricciones `FOREIGN KEY`, y una vista de compatibilidad que reconstruya la relación original mediante una reunión natural.

f) Justificar por escrito, invocando el criterio de superclave sobre el atributo común de la descomposición, por qué la reunión de las dos tablas resultantes es sin pérdida, y migrar los

datos de la instancia de ejemplo hacia las tablas descompuestas, verificando la equivalencia mediante la vista de compatibilidad.

Para poder justificar esto nos basamos en el Teorema de Heath, el cual indica que una descomposición es sin pérdida si el atributo común en ambas tablas es superclave en al menos una de ellas.

En este caso, como podemos ver, el atributo en común es responsable_control_id

En nuestra nueva tabla, responsable_control_id es la clave primaria (superclave), por lo cual, se cumple la condición.

5. Parte 2 — Consulta con desnormalización controlada

5.1. Escenario

El panel de administración de Food Store necesita mostrar, en tiempo real y con actualización frecuente, el **top 5 de categorías por monto vendido en el día**. La consulta que resuelve este reporte hoy recorre `detalle_pedido`, la une con producto, con `categoria` y con `pedido` para filtrar por fecha y por los registros no eliminados, y agrupa y ordena el resultado:

```
SELECT c.nombre AS categoria,
       SUM(dp.subtotal) AS total_vendido
FROM detalle_pedido dp
JOIN producto pr ON pr.id = dp.producto_id
JOIN categoria c ON c.id = pr.categoria_id
JOIN pedido ped ON ped.id = dp.pedido_id
WHERE ped.fecha = CURRENT_DATE
  AND dp.eliminado = FALSE
  AND ped.eliminado = FALSE
GROUP BY c.nombre
ORDER BY total_vendido DESC
LIMIT 5;
```

A medida que la base de Food Store crece, esta consulta —ejecutada muchas veces por minuto por el panel— empieza a demandar un tiempo de respuesta notorio.

5.2. Se pide

a) Ejecutar la consulta anterior con EXPLAIN ANALYZE sobre una instancia de Food Store razonablemente poblada, y reportar el tiempo de ejecución y el nodo del plan que domina el costo.

El tiempo total de ejecución (Execution Time) fue de **144.317 ms**. El nodo del plan que domina al costo es el "Nested Loop" con **143.9 ms**.

b) Elegir **un** patrón de desnormalización controlada (columna precalculada con disparador, o vista materializada) y justificar la elección en no más de un párrafo, respondiendo al menos estas tres preguntas: ¿qué evidencia medida en (a) motiva la decisión?, ¿qué mecanismo garantiza que el dato redundante no se desincronice de su fuente de verdad?, y ¿es reversible sin pérdida de información?

Para optimizar el reporte del top 5 de categorías, se elegí utilizar una vista materializada . Elegí dicho patrón ya que, como pudimos ver con la ayuda del EXPLAIN ANALYZE, el algoritmo Nested Loop actuó como cuello de botella consumiendo 143.9 ms al tener que cruzar repetidamente cuatro tablas transaccionales en cada lectura. Para evitar que este dato redundante se sincronice, utilizó el mecanismo de garantía REFRESH

MATERIALIZED VIEW CONCURRENTLY, el cual exige un índice único, y permite actualizar el resumen en segundo plano sin bloquear dicha consulta. Por último, está desnormalización es totalmente reversible sin pérdida de información, ya que el esquema normalizado original (la fuente de verdad) permanece intacto y bastaría con ejecutar un DROP MATERIALIZED VIEW para regresar al estado inicial.

c) Implementar el patrón elegido con el script SQL completo correspondiente (la estructura desnormalizada y su mecanismo de sincronización).

d) Escribir **al menos una consulta** que resuelva el mismo reporte del punto 5.1 leyendo directamente de la estructura desnormalizada creada en (c), en lugar de recorrer y unir las cuatro tablas originales. Ejecutarla con EXPLAIN ANALYZE y presentar, en una tabla de dos columnas (antes / después), el tiempo de ejecución y el tipo de nodo dominante de cada plan.

	ANTES	DESPUÉS
Tiempo de ejecución	**144.317 ms**	**0.133 ms**
Nodo dominante	**Nested Loop**	**Seq Scan**

e) Escribir un breve script de auditoría que detecte si el dato redundante introducido en (c) se desincronizó de su fuente de verdad, y ejecutarlo sobre la base ya migrada (su resultado debe ser vacío).

6. Entregables

1. `tp_fnbc_control_lote.sql` — esquema original, instancia de ejemplo, tablas descompuestas, vista de compatibilidad y migración de datos verificada (Parte 1).
2. `tp_desnormalizacion_top_categorias.sql` — estructura desnormalizada, su mecanismo de sincronización, la consulta pedida en 5.2(d) y el script de auditoría de 5.2(e) (Parte 2).
3. **Informe breve** (PDF o Word) con: las dependencias funcionales y claves candidatas de la Parte 1, la demostración de violación de FNBC, la justificación de la unión sin pérdida, las capturas de EXPLAIN ANALYZE antes y después de la Parte 2, y la justificación del patrón de desnormalización elegido.7.

Formato de entrega Link con el repositorio de trabajo con los tres artefactos anteriores. El informe y los scripts se evalúan de manera conjunta: una afirmación sin el artefacto que la demuestre no se considera resuelta.